

Proxy HTTP com Filtragem de Conteúdo

Alex Santos¹, Pedro Machado¹

¹Curso de Sistemas de Informação
Universidade Federal do Rio Grande (FURG)
96203-900 – Rio Grande – RS – Brasil

{alex.santos, pedro.machado}@furg.br

Abstract. *This paper describes the implementation of an HTTP proxy with content filtering, developed as a practical assignment for the Internet Systems II course. Built with Python and Flask, the proxy intercepts HTTP requests, blocks listed domains, replaces inappropriate words in HTML pages, and logs every access. The work also covers why content filtering does not apply to HTTPS traffic and what would be needed to support it.*

Resumo. *Este relatório descreve a implementação de um proxy HTTP com filtragem de conteúdo, desenvolvido como trabalho prático da disciplina de Sistemas para Internet II. O projeto foi implementado em Python com Flask, cobrindo as três funcionalidades pedidas: bloqueio de domínios, substituição de palavras em páginas HTML e registro de acessos em log. O relatório também explica por que o filtro não funciona em HTTPS e o que seria necessário para que funcionasse.*

1. Introdução

A proposta do trabalho é construir um proxy: um servidor posicionado entre o navegador e a internet. Em vez de o navegador ir direto ao site, a requisição passa primeiro pelo proxy, que decide entre repassar, bloquear ou modificar o conteúdo antes de entregá-lo.

Proxies desse tipo são encontrados em redes corporativas e escolares para controlar o acesso [Fielding et al. 1999]. O projeto implementa uma versão funcional voltada ao contexto acadêmico, permitindo bloquear domínios configuráveis, substituir palavras indesejadas no HTML e acompanhar os acessos por meio de um log.

2. Tecnologias Utilizadas

A implementação utiliza Python com Flask [Pallets Projects 2026a] para o servidor web, aproveitando a familiaridade com a linguagem trabalhada em aula. A biblioteca `requests` [Reitz 2026] é responsável por realizar as requisições aos sites de destino, enquanto o `BeautifulSoup` [Richardson 2026] trata o HTML antes de devolvê-lo ao navegador. Os arquivos de configuração são JSONs simples, podendo ser editados sem alterar o código.

3. Estrutura do Projeto

O projeto é organizado nos seguintes arquivos:

- **app.py**: código principal contendo as rotas e funções;

- **blocked.json**: lista de domínios bloqueados;
- **words.json**: dicionário de palavras e suas substituições;
- **log.txt**: registro de todos os acessos realizados;
- **templates/**: páginas HTML da interface do proxy renderizadas via Jinja [Pallets Projects 2026b] (início, bloqueio e erro).

4. Como o Proxy Funciona

4.1. Rota principal: /<path:url>

Essa é a rota central do proxy. Ao acessar, por exemplo, `http://localhost:5000/http://exemplo.com`, o Flask captura tudo após a barra como a URL de destino, executando o seguinte fluxo:

1. Extrai o domínio da URL utilizando `urlparse`;
2. Verifica se esse domínio (ou qualquer subdomínio) consta em `blocked.json`;
3. Estando bloqueado, retorna uma página personalizada com status 403, encerrando o fluxo sem contatar o servidor de destino;
4. Não estando bloqueado, realiza a requisição ao site real via `requests`, repassando os headers do cliente e excluindo os que causam conflitos de decodificação, como `accept-encoding`;
5. Sendo a resposta HTML (`Content-Type: text/html`), aplica o filtro de palavras;
6. Injeta a barra de navegação no topo da página via `BeautifulSoup`;
7. Registra o acesso no log e devolve a resposta ao navegador.

4.2. Rota de administração: /

A página inicial funciona como painel de controle, exibindo a lista de sites bloqueados e os palavras cadastrados. Por ela é possível adicionar ou remover entradas de ambas as listas sem editar os JSONs diretamente. Há também um campo para testar o acesso a um domínio direto pela interface.

4.3. Barra de navegação

Em toda página retornada pelo proxy, o `BeautifulSoup` injeta uma barra de navegação no topo, definida em `nav.html`. Ela exibe a URL atual em um campo editável e oferece botões para acessar um novo endereço ou voltar à página inicial.

5. Bloqueio de Domínios

A lógica de bloqueio verifica tanto o domínio exato quanto subdomínios. Tendo `facebook.com` na lista, o acesso a `www.facebook.com` também é bloqueado, utilizando `endswith(". "+dominio_bloqueado)` na comparação.

Ocorrendo o bloqueio, é retornada uma página personalizada indicando o domínio bloqueado, com link para voltar ao início, sem expor mensagens de erro internas.

6. Filtro de Palavrões

O filtro é aplicado somente quando o `Content-Type` da resposta contém `text/html`, evitando corrupção em arquivos JavaScript, CSS ou imagens trafegando pelo proxy.

Para cada entrada de `words.json`, é utilizado `re.sub` com a flag `IGNORECASE`, tratando “Bobão”, “BOBÃO” e “bobão” da mesma forma. Sendo o HTML resultante diferente do original, a ação registrada no log passa a ser `filtrado` em vez de `permitido`.

7. Log de Acessos

Cada requisição gera uma linha em `log.txt` no formato:

```
2026-05-28 20:10:34 | example.com | permitido
2026-05-28 20:11:02 | facebook.com | bloqueado
2026-05-28 20:12:15 | algumsite.com | filtrado
```

O arquivo é aberto em modo *append*, acumulando o histórico entre sessões sem sobrescrever entradas anteriores.

8. Dificuldades e Decisões

A principal dificuldade foi montar a resposta para o Flask após realizar a requisição com `requests`. Não sendo possível repassar o objeto diretamente, foi necessário construir um `Response` de forma explícita, passando conteúdo, status e `Content-Type`. Alguns headers precisaram ser removidos antes de devolver a resposta (`content-encoding`, `content-length`, `transfer-encoding`, `connection`), pois causavam conflitos de decodificação no navegador.

Outro ponto observado durante os testes foi que a maioria dos sites carrega conteúdo via JavaScript após o HTML inicial, tornando o filtro de palavrões ineficaz para esse conteúdo dinâmico. Trata-se de uma limitação inerente a qualquer proxy que inspeciona apenas o HTML estático, devidamente documentada no projeto.

9. Uso de Ferramentas de IA

Ferramentas de IA foram utilizadas em três momentos pontuais: redigindo uma versão inicial do README (depois revisada e adaptada), gerando uma lista inicial de palavrões (expandida manualmente durante os testes) e esclarecendo dúvidas sobre como estruturar o retorno do HTML filtrado, o mecanismo de log e a verificação de `Content-Type`. Em todos os casos, o código foi relido, ajustado ao contexto do projeto e testado antes de chegar à versão final.

10. HTTPS e por que o filtro não funciona

10.1. O problema

Utilizando HTTPS, o tráfego é cifrado com TLS [Dierks and Rescorla 2008]. O proxy consegue identificar o IP do servidor de destino e, via SNI (*Server Name Indication*), o hostname, o que é suficiente para bloquear o domínio. O corpo da resposta, porém, chega completamente cifrado. Sem as chaves da sessão TLS, não há como ler nem modificar o conteúdo trafegado.

10.2. O que seria necessário

Para inspecionar conteúdo HTTPS, o proxy precisaria atuar como MITM (*Man-in-the-Middle*), estabelecendo conexões TLS independentes com o cliente e com o servidor, fazendo a ponte entre as duas. Isso exigiria a geração de certificados dinâmicos assinados por uma CA própria instalada como confiável no dispositivo do usuário, permitindo descriptografar a resposta, aplicar os filtros e recriptografá-la antes de entregá-la.

Ferramentas como o `mitmproxy` utilizam essa abordagem em ambientes de auditoria. O problema é que quem opera o proxy passa a ter acesso a credenciais, dados bancários e mensagens privadas. Sem consentimento explícito do usuário monitorado, isso configura violação grave de privacidade. Esse suporte não foi implementado, mas o mecanismo foi estudado e compreendido.

11. Aprendizados

Implementando o proxy, tornou-se concreto o funcionamento do protocolo HTTP: a estrutura de requisições e respostas, o papel dos headers para o navegador funcionar corretamente, e a diferença entre o HTML entregue pelo servidor e o que é renderizado após a execução do JavaScript.

Ficou evidente também que o navegador busca recursos externos (CSS, imagens, scripts) diretamente do servidor de origem, abrindo conexões independentes para cada um e contornando o proxy. Isso explica por que páginas acessadas via proxy às vezes aparecem sem estilo.

12. Conclusão

O proxy implementado atende aos requisitos propostos: repassando conteúdo, bloqueando domínios com página personalizada, filtrando palavras no HTML e registrando os acessos com timestamp e ação. As limitações identificadas, como o carregamento dinâmico via JavaScript, os recursos externos que não passam pelo proxy e a ausência de suporte a HTTPS, são inerentes à abordagem adotada e estão devidamente documentadas.

Referências

- Dierks, T. and Rescorla, E. (2008). The transport layer security (TLS) protocol version 1.2. RFC 5246, IETF.
- Fielding, R., Gettys, J., Mogul, J., Frystyk, H., Masinter, L., Leach, P., and Berners-Lee, T. (1999). Hypertext transfer protocol – HTTP/1.1. RFC 2616, IETF.
- Pallets Projects (2026a). Flask documentation. Disponível em: <https://flask.palletsprojects.com>. Acesso em: maio 2026.
- Pallets Projects (2026b). Jinja documentation. Disponível em: <https://jinja.palletsprojects.com>. Acesso em: maio 2026.
- Reitz, K. (2026). Requests: HTTP for humans. Disponível em: <https://docs.python-requests.org>. Acesso em: maio 2026.
- Richardson, L. (2026). Beautiful soup documentation. Disponível em: <https://www.crummy.com/software/BeautifulSoup/bs4/doc>. Acesso em: maio 2026.